



Modificação de Valores em Python

Atribuição e Estado do Programa

Conceito

Atribuição associa um nome a um valor na memória. Quando um valor muda, o estado do programa muda. Valores antigos não são lembrados automaticamente - o programa evolui conforme cada linha é executada.

Conceito-chave: Cada atribuição substitui o valor anterior, a menos que você o salve em outra variável.

Exemplo

```
x = 10
x = x + 5 # o valor anterior (10) não é preservado
print(x) # 15
```



Saída real:

15

Execução Sequencial

Ordem Importa

As instruções são executadas de cima para baixo, linha por linha.

```
a = 5  
b = a * 2  
a = 10  
print(b) # 10, não 20
```

Estado Atual

Cada linha depende do estado atual das variáveis no momento da execução.

Resultado Final

Trocar a ordem das linhas muda completamente o resultado.



Saída real:

10

O valor de b é calculado quando a = 5, então permanece 10 mesmo após a mudança em a.

Carregando Dados de uma URL

Dados podem ser carregados diretamente de uma URL sem precisar baixar o arquivo manualmente. O arquivo remoto é lido e convertido em uma tabela pandas, ficando disponível para análise imediata.

```
import numpy as np
import pandas as pd
url = "https://raw.githubusercontent.com/eogasawara/tutorial-python/main/examples/baralho.csv"
baralho = pd.read_csv(url)
baralho.head()
```



Saída real:

```
face naipe
0 ás ouros
1 dois ouros
2 três ouros
3 quatro ouros
4 cinco ouros
```

Salvando Dados em Arquivo Local

Tabela na Memória

Dados carregados e processados

Remover a Variável

Apaga o nome baralho

```
baralho.to_csv("baralho.csv", index=False)  
del baralho
```

Salvar em Disco

Persiste o estado atual

Arquivo Disponível

Permanece no diretório

O parâmetro `index=False` evita salvar a coluna de índice. O comando `del` apaga o nome `baralho`; a memória é liberada quando não há outras referências ao objeto. O arquivo permanece no disco.

Recarregando Dados do Arquivo

Após salvar, os dados podem ser recarregados do disco a qualquer momento, exatamente como estavam no salvamento. O arquivo local funciona como armazenamento persistente, permitindo interromper e retomar análises.

```
import pandas as pd
baralho = pd.read_csv("baralho.csv")
baralho.head()
```



Saída real:

face naipe

0 ás ouros

1 dois ouros

2 três ouros

3 quatro ouros

4 cinco ouros

Adicionando Colunas à Tabela

Uma nova coluna é criada atribuindo valores diretamente à tabela. O tamanho do vetor deve corresponder ao número de linhas e pode ser usado em operações subsequentes.

Função `numpy.arange`: Cria uma sequência numérica de 1 até o tamanho da tabela.

```
baralho["idx"] = np.arange(1, len(baralho) + 1)  
baralho.head()
```



Saída real:

face naipe idx

0 ás ouros 1

1 dois ouros 2

2 três ouros 3

3 quatro ouros 4

4 cinco ouros 5

Alterando Valores por Índice

Selecionar Posições

Use listas de índices para escolher quais elementos modificar

```
baralho.loc[[0, 2, 4], "idx"] = 1  
baralho.head()
```

Atribuir Novos Valores

A modificação é feita diretamente na coluna especificada

Verificar Resultado

As mudanças aparecem imediatamente na tabela



Saída real:

face naipe idx

0 ás ouros 1

1 dois ouros 2

2 três ouros 1

3 quatro ouros 4

4 cinco ouros 1

Esse método é usado para correções pontuais em valores específicos da tabela.

Alterando Valores por Intervalo

Faixas de posições podem ser modificadas em bloco usando intervalos. Operações matemáticas são aplicadas aos valores existentes, facilitando ajustes sistemáticos.

```
baralho.loc[3:5, "idx"] = baralho.loc[3:5, "idx"] + 1  
baralho.loc[0:6, ["idx"]]
```

❓ **Saída real:**

idx

0 1

1 2

2 1

3 5

4 2

5 7

6 7

Nota importante: Em pandas, intervalos com `loc` incluem o último elemento (diferente do Python padrão).

Criando Vetores Lógicos

Comparações geram vetores booleanos (True/False) que indicam quais linhas satisfazem uma condição. Operações matemáticas ajudam a criar critérios complexos. Esses vetores são a base da filtragem de dados.

Operação Matemática

Módulo % verifica divisibilidade

```
vec = (baralho["idx"] % 2 == 1)
idx = baralho.loc[vec, "idx"]
idx.head()
```

Comparação

== 1 identifica valores ímpares

Resultado Booleano

Vetor de True/False para cada linha



Saída real:

0 1

2 1

3 5

5 7

6 7

Name: idx, dtype: int64

Aplicando Filtros

Vetores booleanos selecionam linhas da tabela de forma eficiente. Somente as linhas onde o vetor é True são mantidas no resultado, produzindo subconjuntos dos dados baseados em critérios específicos.

```
cartas = baralho[vec]  
cartas.head()
```



Saída real:

face naipe idx

0 ás ouros 1

2 três ouros 1

3 quatro ouros 5

5 seis ouros 7

6 sete ouros 7

O resultado é uma nova tabela filtrada contendo apenas as linhas que atendem à condição original.

Operações de Comparação

Comparação Simples

Retorna um único booleano

```
1 > 2
```

 **Saída:** False

Comparação Vetorial

Compara elemento por elemento

```
np.array([1, 2, 3]) == np.array([3, 2, 1])
```

 **Saída:** array([False, True, False])

Verificação de Pertinência

Testa se elementos estão em conjunto

```
np.isin([1, 2, 3], [3, 4, 5])
```

 **Saída:** array([False, False, True])

Operadores de comparação produzem valores booleanos. Quando aplicados a vetores, funcionam elemento a elemento, permitindo comparar estruturas inteiras de uma vez. O resultado é usado em operações de filtragem.

Operadores Lógicos

Operador AND (&)

Exige que ambas as condições sejam verdadeiras simultaneamente. Útil para filtros restritivos.

Exemplo com AND

```
x = (baralho["face"] == "dama") & (baralho["naipe"] == "espadas")  
baralho[x].head()
```



Saída:

```
face naipe idx  
50 dama espadas 51
```

Operador OR (|)

Aceita quando pelo menos uma condição é verdadeira. Útil para filtros mais abrangentes.

Exemplo com OR

```
y = (baralho["face"] == "dama") | (baralho["naipe"] == "espadas")  
baralho[y].head()
```



Saída:

```
face naipe idx  
11 dama ouros 12  
24 dama copas 25  
37 dama paus 38  
39 ás espadas 40  
40 dois espadas 41
```

Trabalhando com Valores Ausentes

Valores ausentes são representados por NaN (Not a Number). Operações com NaN normalmente produzem NaN, mas funções estatísticas específicas podem ignorar esses valores ausentes, evitando resultados inválidos.

Operação Padrão

`np.mean(v)` retorna NaN se houver ausentes

```
v = np.array([np.nan] + list(range(1, 51)))  
np.mean(v)  
np.nanmean(v)
```

Operação Segura

`np.nanmean(v)` ignora NaN e calcula a média dos valores válidos



Saídas reais:

```
nan  
25.5
```

Filtrando com Dados Ausentes

Comparações com valores ausentes retornam False ou NaN, podendo eliminar linhas inesperadamente. Trate os ausentes antes de filtrar para evitar erros sutis na análise.

```
# Criando coluna "valor" a partir de "face" (ás=1 ... rei=13)
ordem = ["ás", "dois", "três", "quatro", "cinco", "seis", "sete", "oito", "nove", "dez", "valete", "dama", "rei"]
mapa = {face: i+1 for i, face in enumerate(ordem)}
baralho["valor"] = baralho["face"].map(mapa)

# Introduzindo alguns NaN (um por naipe)
baralho.loc[[0, 13, 26, 39], "valor"] = np.nan
```

```
baralho[baralho["valor"] < 3]
```

❏ Saída (filtro valor < 3):

```
face naipe idx valor
1 dois ouros 2 2.0
14 dois copas 15 2.0
27 dois paus 28 2.0
40 dois espadas 41 2.0
```

```
baralho["valor"].isna().sum()
```

❏ Contagem de NaN:

4

Use `isna()` para identificar valores ausentes e `dropna()` ou `fillna()` para tratá-los.

Referências

Think Python

Downey, A. *Think Python: How to Think Like a Computer Scientist*. O'Reilly Media.

Python Data Science Handbook

VanderPlas, J. *Python Data Science Handbook*. O'Reilly Media.

Data Science from Scratch

Grus, J. *Data Science from Scratch*. O'Reilly Media.